

Chapter 1

Github Repository Link - <https://github.com/reim7/CS4080--HW1>

1. There are at least six domain-specific languages used in the **little system I cobbled together** to write and publish this book. What are they?

The six domain-specific languages used in the little system cobbled together to write and publish this book are Markdown, Jinja2, Makefile, SASS, CSS, and HTML. There are also custom tags added throughout the code and Markdown that help connect the two together.

2. Get a “Hello, world!” program written and running in Java. Set up whatever makefiles or IDE projects you need to get it working. If you have a debugger, get comfortable with it and step through your program as it runs.

In the repository.

3. Do the same thing for C. To get some practice with pointers, define a **doubly linked list** of heap-allocated strings. Write functions to insert, find, and delete items from it. Test them.

In the repository.

Chapter 2

1. **Pick an open source implementation of a language you like. Download the source code and poke around in it. Try to find the code that implements the scanner and parser. Are they handwritten, or generated using tools like Lex and Yacc? (.l or .y files usually imply the latter).**

I chose CPython, the open-source implementation of the Python programming language. After looking through the source code, I found that the grammar used by the parser is mainly defined in the Grammar/python.gram file.

CPython uses a PEG (Parsing Expression Grammar) parser. The parser is generated from the grammar rules in python..gram using Python's own PEG parser rather than traditional tools like Lex or Yacc. I did not find .l or .y files being used for the main python parser.

The scanner or tokenizer is implemented separately in the parser/lexer and tokenizer related source files. CPython, unlike a traditional lex-generated scanner, implements its own source code.

Overall, CPython uses a combination of handwritten tokenization code and a generated PEG parser rather than using Lex and Yacc.

2. **Just-in-time compilation tends to be the fastest way to implement dynamically typed languages, but not all of them use it. What reasons are there to *not* JIT?**

The reasons not to use JIT include that it is really complex to implement, troubleshoot, and maintain, and relatively few people have the expertise needed to do it effectively. Similar to a native code compiler, it limits you to a particular CPU architecture. Bytecode is usually more compact than machine code because it more closely represents the language's semantics. As a result, it requires less memory. On systems such as embedded devices, where conserving memory may be more important than maximizing speed which could be a useful trade-off. Certain platforms, including iOS and most gaming consoles don't allow code generated at runtime to be executed.

3. **Most Lisp implementations that compile to C also contain an interpreter that lets them execute Lisp code on the fly as well. Why?**

Most Lisp languages support Macros, which are pieces of code that run during compile time. Because of this, the implementation needs to be able to evaluate a macro while the rest of the program is still being compiled. One option would be to compile the macro first then execute it, but doing so would require a significant amount of extra work and overhead.

Chapter 3

1. **Write some sample Lox programs and run them (you can use the implementations of Lox in [my repository](#)). Try to come up with edge case behavior I didn't specify here. Does it do what you expect? Why or why not?**

For my sample Lox program, I created a counter class that stores a value and provides methods for increasing, decreasing, and displaying it. I tested the class with positive numbers, zero, and negative numbers.

One edge case I tested was decreasing a counter whose value was already zero. I wanted to see whether Lox would allow the value to become negative or produce an error. The program allowed the value to change from 0 to -1.

This behavior makes sense because Lox treats the value as a number and the counter class does not contain any condition that prevents negative values. Lox therefore performs the subtraction normally. If I wanted the counter to stop at zero, I would need to add that restriction myself inside the `decrease()` method.

2. **This informal introduction leaves a *lot* unspecified. List several open questions you have about the language's syntax and semantics. What do you think the answers should be?**

Some questions I would have would be what happens if you declare two classes or functions with the same name, can a class inherit from something that isn't a class, can you reassign to the name that is bound by a class or function declaration, and what kind of expressions are allowed when a superclass is specified. I need to learn more about Lox before I am able to think about their answers.

3. **Lox is a pretty tiny language. What features do you think it is missing that would make it annoying to use for real programs? (Aside from the standard library, of course.)**

The features that I think are missing that would make it annoying to use for real programs are arrays and built in lists, switch statements, and break and continue loops.

Lox does not have a built-in array type, although it is possible to create something like a linked list using classes, there is no simple way to store a sequence of values and quickly access an

element by its index. Having arrays or another indexed collection type would make working with larger amounts of data much easier.

Los could also benefit from a way to handle runtime errors. Features like try, catch, and throw would allow programs to recover from certain errors instead of simply stopping when something goes wrong.

Lox does not have switch statements either, when a program needs to compare one value against several possibilities, we have to use a long chain of if else statements..